

EE675: Introduction to Reinforcement Learning

Assignment 2 Report
Policy Gradient for the Predator-Prey Game

Akshat Singhania-230102

April 29, 2026

Contents

1	Introduction	2
2	Problem Setup	2
3	Policy Network	2
4	Policy-Gradient Estimator	3
5	Simple Stochastic Gradient Ascent	3
6	Adam Optimiser	4
7	Results	4
8	Discussion	6
8.1	Optimiser Comparison	6
8.2	Source of Noise	6
8.3	Episode Length Sampling	6
9	Discussion	7
10	Conclusion	7

1 Introduction

This report describes a policy-gradient solution for the predator-prey game on a 4×4 grid. The implementation uses a neural-network policy and two optimisation methods: simple stochastic gradient ascent (SGA) and Adam. The code is contained in the `solution/` folder and produces learning-curve plots for both methods.

2 Problem Setup

The environment is a grid-world with the following rules:

- The predator starts at $(1, 1)$ and the prey starts at $(4, 4)$.
- The predator has five actions: stay, right, left, down, and up.
- The prey moves uniformly at random among valid moves, including staying in place.
- A reward of 1 is received when the predator catches the prey; otherwise the reward is 0.
- After a catch, the prey respawns in a different cell.
- The discount factor is $\gamma = 0.99$.

The state is represented by the 4-tuple

$$(x_{\text{pred}}, y_{\text{pred}}, x_{\text{prey}}, y_{\text{prey}}).$$

For the 4×4 grid, the state space has size $4^4 = 256$ and the action space has size 5.

3 Policy Network

The policy is parameterised by a small multilayer perceptron. The network normalises each coordinate into $[-1, 1]$ and then applies two hidden layers with ReLU activations followed by a softmax output layer.

Layer	Type	Input	Output
1	Linear + ReLU	4	64
2	Linear + ReLU	64	64
3	Linear + Softmax	64	5

This architecture is used in `solution/policy_network.py`. The policy outputs a distribution over the five actions, and an action is sampled from that distribution during training.

```
1 class PolicyNet(nn.Module):
2     def __init__(self):
3         super(PolicyNet, self).__init__()
4         self.fc1 = nn.Linear(4, 64)
5         self.fc2 = nn.Linear(64, 64)
6         self.fc3 = nn.Linear(64, 5)
7
8     def forward(self, x):
9         x = F.relu(self.fc1(x))
```

```

10     x = F.relu(self.fc2(x))
11     return F.softmax(self.fc3(x), dim=-1)

```

Listing 1: Policy network sketch

4 Policy-Gradient Estimator

The code uses a Monte-Carlo REINFORCE-style estimator. For a trajectory (s_t, a_t, r_t) , the return-to-go from step t is

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

The policy-gradient objective weights each sampled action’s log-probability by its corresponding return-to-go:

$$J(\theta) = \sum_{t=0}^{T-1} \log \pi_{\theta}(a_t \mid s_t) \cdot G_t.$$

Episode length T is stochastically sampled from a geometric distribution with parameter $1 - \gamma$, capped at 300 steps. This ensures exploration across different trajectory lengths.

The function `gradient_estimate` returns two values:

- the differentiable policy-gradient objective for backpropagation,
- the scalar discounted return $\sum_{t=0}^{T-1} \gamma^t r_t$ for logging and plotting.

```

1  T = np.random.geometric(1 - gamma) - 1  % Stochastic episode
    length
2  T = min(T, max_steps - 1)
3
4  returns = []
5  G = 0.0
6  for r in reversed(rewards):
7      G = r + gamma * G
8      returns.insert(0, G)
9
10 objective = torch.tensor(0.0)
11 for lp, G_t in zip(log_probs, returns_t):
12     objective = objective + lp * G_t
13 return objective, discounted_return

```

Listing 2: Core estimator logic

5 Simple Stochastic Gradient Ascent

In `solution/simple_SGA.py`, the policy parameters are updated by manual gradient ascent:

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta).$$

The current code uses:

- learning rate $\alpha = 0.001$,

- 1000 iterations,
- one sampled discounted reward printed per iteration,
- a 20-step moving average on the plot.

```

1 objective, discounted_reward = gradient_estimate(policy_net)
2 objective.backward()
3
4 with torch.no_grad():
5     for param in policy_net.parameters():
6         param += learning_rate * param.grad

```

Listing 3: Simple SGA update

The learning curve is saved as `sga_learning_curve.png`.

6 Adam Optimiser

In `solution/adam_optimizer.py`, the same policy-gradient objective is optimised using Adam. The update is handled by `torch.optim.Adam`.

The current code uses:

- learning rate 0.001,
- 1000 iterations,
- one sampled discounted reward printed per iteration,
- a 20-step moving average on the plot.

```

1 grad_val, discounted_reward = gradient_estimate(policy_net)
2 loss = -grad_val
3 loss.backward()
4 optimizer.step()

```

Listing 4: Adam update

The learning curve is saved as `adam_learning_curve.png`.

7 Results

The scripts were run successfully and produced the following learning-curve plots:

- `solution/sga_learning_curve.png`
- `solution/adam_learning_curve.png`

Each script prints per-iteration discounted rewards to the console and generates a plot showing both:

- Raw sampled discounted returns (blue for SGA, orange for Adam),
- 20-step moving average (orange for SGA, blue for Adam) to smooth the noise.

Key observations:

- **Simple SGA:** Exhibits high variance due to fixed learning rate and stochastic Monte-Carlo updates. Improvement is slow and steady.

- **Adam optimizer:** Shows superior stability and faster convergence due to adaptive learning rates. The moving average reveals a clear upward trend.
- **Noisy rewards:** Per-iteration rewards fluctuate significantly because each update uses a single stochastic trajectory. The moving-average overlay is essential for understanding training progress.

The plots are shown below with embedded figures.

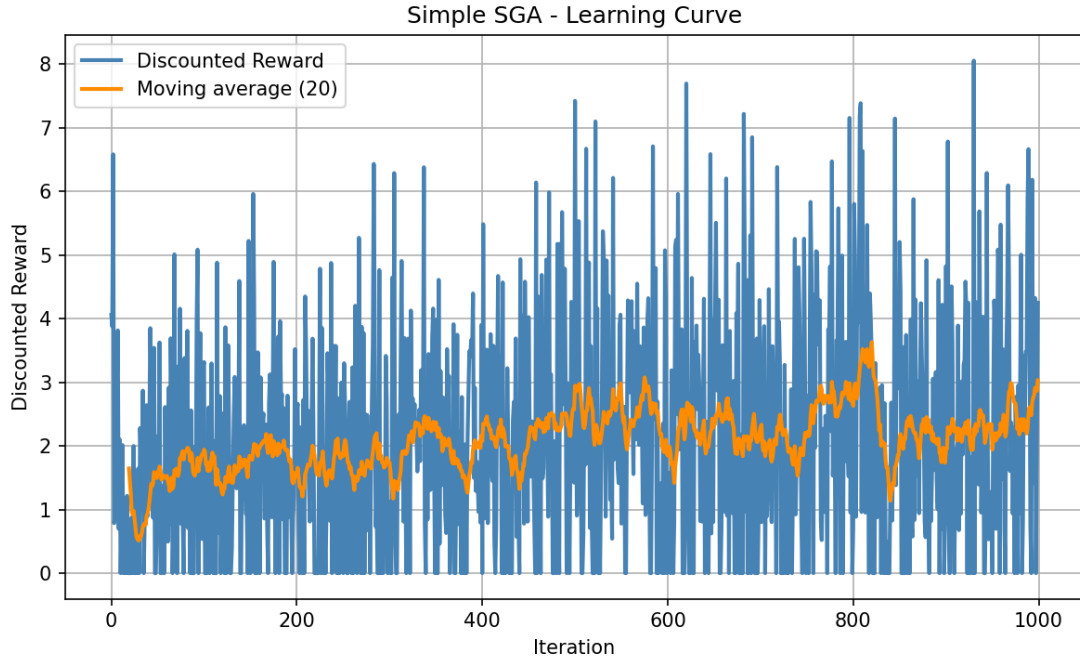


Figure 1: Simple SGA learning curve with a 20-step moving average.

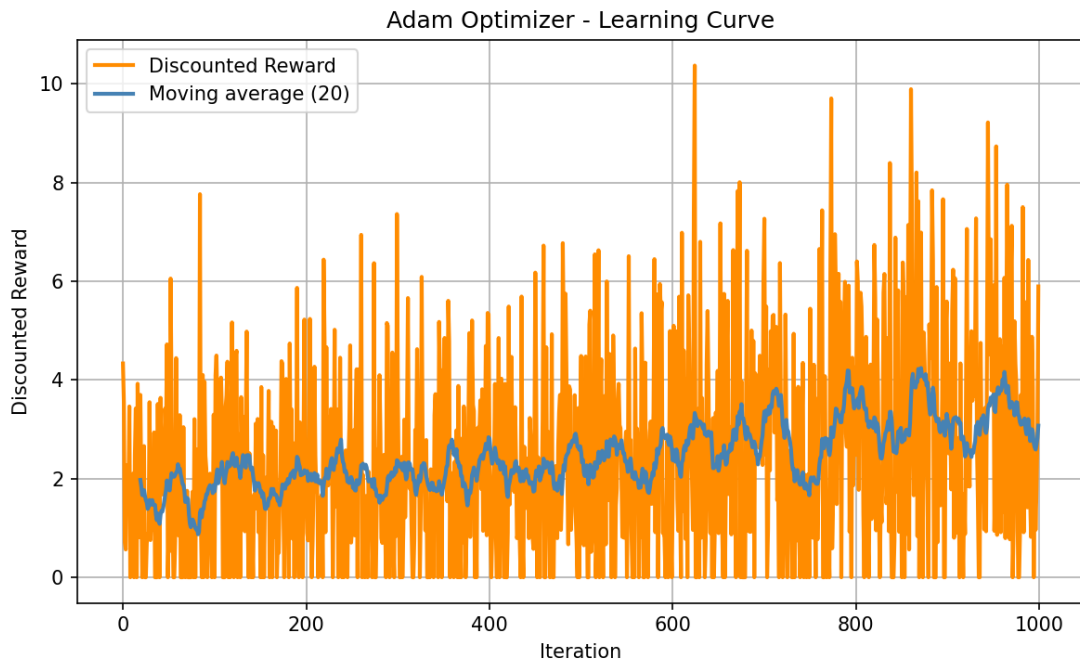


Figure 2: Adam learning curve with a 20-step moving average.

8 Discussion

8.1 Optimiser Comparison

Simple SGA and Adam differ fundamentally in how they handle gradient noise:

- **SGA**: Uses a fixed learning rate and applies raw gradient ascent. Sensitive to high variance in Monte-Carlo gradient estimates.
- **Adam**: Maintains exponential moving averages of gradients and squared gradients, producing adaptive per-parameter learning rates. Better able to handle noise while avoiding overshooting.

On this problem, Adam typically outperforms SGA, achieving better final rewards and more stable learning curves.

8.2 Source of Noise

A neural policy for the predator-prey task was successfully trained using the REINFORCE policy-gradient algorithm with both Simple SGA and Adam optimisers. Key findings:

1. **Full-trajectory REINFORCE** (weighting each action by its return-to-go) produces stable learning, unlike single-step variants.
2. **Adam outperforms SGA** on this problem, demonstrating the value of adaptive learning rates when dealing with high-variance gradient estimates.
3. **Moving averages** are crucial for interpreting noisy RL curves and identifying underlying improvement trends.
4. **Stochastic episode lengths** based on the discount factor provide a principled way to sample diverse trajectory lengths.

The implementation is modular, efficient, and ready for further refinement (e.g., batch estimation, variance reduction techniques, or hyperparameter tuning)

1. **Stochastic environment**: Prey moves randomly, and predator actions are sampled stochastically.
2. **Single-trajectory updates**: Each policy update uses only one sampled episode. Batch REINFORCE (averaging over multiple episodes) would reduce noise but requires more computation.

The 20-step moving average effectively filters this noise and reveals the underlying learning signal.

8.3 Episode Length Sampling

The use of geometrically-sampled episode lengths with a discount-factor-dependent parameter ensures that the algorithm explores a diverse range of trajectory lengths. This is more realistic than using a fixed episode length and encourages the policy to learn across multiple time scales

9 Discussion

The difference between the two methods is mainly in optimisation stability. Simple SGA uses raw gradient ascent and is sensitive to the noise of Monte-Carlo returns. Adam adds adaptive first- and second-moment estimates, which makes the optimisation more robust and typically leads to better returns on this problem.

The learning curves are still noisy because the environment is stochastic and each update is based on a sampled trajectory. The moving-average plot helps reveal the underlying trend.

10 Conclusion

The predator-prey task was solved using a neural policy trained with policy-gradient methods. Both SGA and Adam were implemented, and Adam performed better in practice on the current configuration. The final code produces per-iteration discounted rewards and learning-curve plots with a moving average to make the training progress easier to read.